MODULE 29 · DOMAIN 4 - OPERATING, MONITORING, SECURING

Drift and model monitoring

Four monitor types, baselines and violations, and what each kind of drift means.

MODULE 29 of 43 DOMAIN 4 - Monitoring WEIGHT 24%

admin@sthithapragna.com

A model does not degrade because it broke. It degrades because **the world changed**.

Model Monitor is closed to new customers

- **Closed to new customer access, effective 30 July 2026.** AWS states the date explicitly.
- Existing customers continue as normal. Security and availability work continues; no new features.
- It remains named in the exam guide, so it remains **examinable**.
- **The concepts are the exam.** Baselines, constraints, violations and the four monitor types are generic MLOps.
- The sixth maintenance flag in the course, on the same date as the wider wave.

> ON THE EXAM
>
> Same pattern as Clarify: the mechanism is closed, the mathematics is not. Statistical drift detection is public work.

Three kinds of drift

KIND	WHAT CHANGED
Data drift	The input distribution changed. P(X) moved
Concept drift	The relationship between inputs and outcome changed. P(Y given X) moved
Label drift	The distribution of the outcome itself changed. P(Y) moved

ON THE EXAM

These are different problems with different fixes, and only one of them is visible without labels.

Data drift, and why it is the easy one

- The statistical properties of the input features change over time.
- **Detectable immediately**, because you only need the inputs. No labels required.
- Detected by comparing distributions against a baseline: means, ranges, missing rates, category frequencies.
- Distance measures such as **chi-squared**, **L-infinity** and **JS divergence** quantify the change.
- Data drift does **not** prove the model got worse. It proves the inputs changed.

> ON THE EXAM
>
> Data drift is a leading indicator, not a verdict. It tells you to look, not that something is broken.

Concept drift, and why it is the hard one

- The **relationship** between features and outcome changes. The same input now means something different.
- **Inputs can look completely unchanged** while the model becomes wrong.
- So data quality monitoring **cannot detect it**. Only comparing predictions to outcomes can.
- Fraud is the standard example: attackers change tactics, so old signals stop meaning fraud.
- It needs **ground truth**, which arrives late, which is why it is discovered late.

> ON THE EXAM
>
> This is the one that hurts. Your input monitoring is green, your dashboards are clean, and the model is quietly wrong.

Data drift or concept drift?

DATA DRIFT

Inputs moved

- Detectable from inputs alone.
- Visible within hours.
- May not affect accuracy at all.
- Retraining on recent data usually helps.

CONCEPT DRIFT

Meaning moved

- Needs outcomes to detect.
- Visible only when labels arrive.
- Accuracy falls with unchanged inputs.
- Retraining needs NEW labels, not new features.

THE TIEBREAKER

If accuracy fell and the input distributions look normal, that is concept drift. The absence of data drift is evidence, not reassurance.

The four steps AWS describes

01
Baseline
From the training data. Computes metrics, suggests constraints.

›

02
Capture
Data capture records production requests and responses.

›

03
Schedule
A monitoring job runs on a cadence and compares.

›

04
Violate
Anything outside the constraints is reported as a violation.

›

05
Alarm
Metrics reach CloudWatch, where alarms trigger action.

ON THE EXAM

Every monitoring tool has this shape. The baseline is what 'normal' means, and everything after is comparison against it.

Data capture, configured properly

- **Nothing here works without data capture.** It is off by default, from module 20.
- Configure a **sampling percentage**. At high volume you rarely need every request.
- **It cannot be applied retrospectively.** Yesterday's traffic is gone if capture was off.
- Capture both request and response, since monitors compare inputs and outputs.
- Captured payloads inherit the data's sensitivity. Module 08's protections apply to the capture bucket.

> ON THE EXAM
>
> Enable capture on the day you deploy, not on the day you need it. That is the whole of the advice.

Monitor the data pipeline, not just the model

- Most production model failures start as **data failures** further upstream.
- A schema change, a late job, a changed unit, a source system upgrade. Module 03 and module 04.
- **Glue Data Quality** rules on the pipeline catch it before it reaches the model at all.
- Feature pipelines from module 28 deserve their own monitoring, separate from the model's.
- Catching it at the model is catching it late: predictions have already been served.

> ON THE EXAM
>
> The earlier in the pipeline you detect a data problem, the fewer wrong predictions it produced before you noticed.

Baselines and constraints

- A **baselining job** profiles the training data and produces statistics plus suggested constraints.
- **Statistics** describe the data. **Constraints** are the rules a violation is measured against.
- AWS **suggests** constraints, and you can tighten or loosen them. They are a starting point.
- If a baselining job exists, a monitor **picks up its analysis configuration automatically**.
- **A stale baseline is worse than none.** If the world legitimately moved, your baseline is now fiction.

> ON THE EXAM
>
> Module 17 registered five baselines on a model package. This is where those registered baselines are actually used.

When to re-baseline

- **After a retrain**, because the new model's expected behaviour differs from the old one's.
- **After a legitimate, accepted shift** in the data that is not going to reverse.
- **Not** because the monitor keeps firing. That is the monitor doing its job.
- Re-baselining to silence an alarm converts a detection system into a confirmation system.
- Record why each baseline was taken. An auditor will ask what normal meant, and when.

> ON THE EXAM
>
> The temptation is real: an alarm that fires every day is annoying, and re-baselining makes it stop. It also makes it useless.

Thresholds, and alert fatigue

- **Too tight** and it fires constantly. People stop reading it within a fortnight.
- **Too loose** and it never fires, which looks identical to everything being fine.
- Start from the **natural variation** in your data, measured over a period when nothing was wrong.
- Separate **severity**: something to look at weekly, and something to wake somebody for.
- Review what fired last quarter and what you did about it. That is how thresholds get calibrated.

> ON THE EXAM
>
> An alarm nobody reads is worse than no alarm, because it creates the belief that somebody is watching.

What each monitor watches

MONITOR	WHAT IT COMPARES, AND WHAT IT NEEDS
Data quality	Input distributions against a baseline. Needs inputs only
Model quality	Predictions against ground truth. Needs labels
Bias drift	Fairness metrics over time. Needs labels. Clarify-powered
Feature attribution drift	Which features drive predictions. Needs features only. Clarify-powered

ON THE EXAM

Two need labels and two do not. That split decides what you can monitor on day one and what has to wait.

Data quality monitoring

- Compares the statistical properties of incoming requests against the baseline constraints.
- Checks distribution changes, **missing values, data types and feature ranges**.
- **Available immediately**, because it needs no outcomes. Start here on any new deployment.
- Violations land in a **constraint_violations.json** file alongside the report.
- It catches broken pipelines as readily as genuine drift. Both are worth catching.

> ON THE EXAM
>
> A column that silently became null is a data quality violation, and it is far more common than genuine distributional drift.

Model quality monitoring

- Compares predictions against **actual ground truth labels** you store in S3.
- A **merge job** joins captured inference data with those labels before comparing.
- Metrics depend on the **problem type**: accuracy and F1 for classification, RMSE and MAE for regression.
- This is the only monitor that measures whether the model is actually still **right**.
- It requires labels, which is why many teams never enable it.

> ON THE EXAM
>
> Module 17's metrics, running continuously on production traffic. The metric you chose there is the one that degrades here.

Ground truth arrives late, so use offsets

- **Ground truth labels can be delayed**, sometimes by weeks. A loan default is known months later.
- So you **specify offsets** when creating the monitoring schedule, to align predictions with labels.
- The offset says how far back to look for the predictions that these labels correspond to.
- Without it, the merge finds no matching labels and the monitor reports nothing useful.
- The delay also sets your **detection floor**: you cannot find concept drift faster than labels arrive.

> ON THE EXAM
>
> The label delay is the hard limit on how quickly quality degradation can ever be detected. Design around it, not against it.

Bias drift and feature attribution drift

- **Bias drift** watches module 19's post-training bias metrics change over time. It needs labels.
- **Feature attribution drift** watches which features drive predictions, using a **SHAP baseline**.
- **Attribution drift needs features only**, so ground truth labelling is excluded. No labels required.
- That makes attribution drift available immediately, like data quality monitoring.
- A shift in feature importance is an **early warning**: reasoning changes before accuracy does.

> ON THE EXAM
>
> Feature attribution drift is the underused one. It detects a changing model without waiting for labels, which is rare and valuable.

Model Monitor works on tabular data

- **Model Monitor computes metrics and statistics on tabular data only.** AWS states this plainly.
- An image classification model can still be monitored, but only on its **output**.
- It cannot compute statistics on the images themselves. There is no distribution to profile.
- So for vision and text models, you monitor **predicted class distribution and confidence**, not the input.
- Monitoring unstructured input drift means embedding it yourself and profiling the vectors.

> ON THE EXAM
>
> This is a clean exam fact and a real constraint. For an image model, 'monitor the inputs' is not something Model Monitor can do.

What a monitoring plan should state

QUESTION	WHY IT MUST BE ANSWERED IN ADVANCE
What are we monitoring?	Which inputs, outputs, segments and business metrics
How often?	The cadence, and therefore the cost
What is the threshold?	And what natural variation it was derived from
Who is alerted?	A named team or rota, not an inbox
What is the response?	Retrain, investigate, or accept. Decided before it fires
When do we re-baseline?	So it is a decision rather than a reaction to noise

ON THE EXAM

Six questions. A monitoring setup that cannot answer them will be switched off within a year, usually during a cost review.

Monitoring schedules

- A schedule defines **what data to collect, how often, how to analyse it, and which reports to produce**.
- Cadence is a cron expression. Hourly is common; daily suits slower-moving data.
- **You specify either a batch transform input or an endpoint input, but not both.**
- Reports, statistics and violations go to S3. Metrics go to CloudWatch.
- Each execution is a processing job, so a frequent schedule has a real and recurring cost.

> ON THE EXAM
>
> The cadence is a cost decision as much as a detection one. Hourly monitoring of a slow-moving model is an expensive habit.

Monitoring batch transform

- Batch transform jobs can be monitored too, using a **batch transform input** on the schedule.
- Remember the constraint: **either a batch transform input or an endpoint input, not both**.
- So a model served both ways needs **two schedules**, one per serving path.
- Batch is often where the largest volume runs, so it deserves monitoring at least as much.
- Teams monitor the endpoint and forget the nightly batch job scoring ten million records.

> ON THE EXAM
>
> The forgotten batch job is a real pattern. It is usually the higher-volume path and usually the less-watched one.

From violation to action

- Metrics reach **CloudWatch**, where you set alarms on thresholds that matter.
- An alarm can trigger **remedial action**: notify a person, or start a retraining pipeline.
- That is module 28's drift trigger, and it is the most defensible retraining trigger available.
- **A violation nobody acts on is a log entry.** Decide the response before you enable the monitor.
- Not every drift warrants a retrain. Some warrant a look, and some warrant nothing.

> ON THE EXAM
>
> Module 28 asked what should trigger a retrain. This is the answer, and it only works if somebody defined what to do.

Responding to a drift alarm

- **Confirm it is real.** Check whether an upstream job failed before assuming the world changed.
- **Scope it.** Which features, which segments, since when? Module 19's lineage answers when.
- **Decide whether it matters.** Drift with stable accuracy and stable business outcomes may not.
- Then act: retrain, re-baseline, fix the pipeline, or accept and document.
- Write the runbook before the alarm, not during the incident.

> ON THE EXAM
>
> Four steps: confirm, scope, judge, act. Most teams skip straight to retraining, which is expensive and often the wrong move.

Lineage during an incident

- **Which model version served this prediction?** The registry and endpoint history answer that.
- **Which data trained it?** Lineage from module 19, if it was recorded at the time.
- **What did the input look like?** Data capture, if it was enabled.
- Those three together let you reconstruct a decision. Missing any one makes it guesswork.
- None of them can be added retrospectively once the incident has happened.

> ON THE EXAM
>
> An incident review is a lineage query. Everything you did not record in advance is a question you cannot answer.

Responses to drift

FINDING	SENSIBLE RESPONSE
Data quality violation on one feature	Check the pipeline first. It is usually upstream, not the world
Gradual data drift, accuracy holding	Note it. Re-baseline. Do not retrain yet
Data drift with accuracy falling	Retrain on recent data
Accuracy falling, no data drift	Concept drift. Retrain, and you need new labels
Bias drift on a protected attribute	Investigate before retraining. Module 08 and 19
Feature attribution shifted	Early warning. Look before accuracy confirms it

ON THE EXAM

Row one is the most common by a wide margin. Most 'drift' is a broken upstream job, and that is a cheaper fix than retraining.

Monitor by segment, not only overall

- Overall metrics can look stable while one segment degrades badly.
- Break monitoring down by **region, channel, customer type, device**, and any protected attribute.
- A new segment is the most likely to drift, because the model saw least of it in training.
- This is module 17's segment evaluation, run continuously instead of once.
- Report the worst segment alongside the aggregate, as module 17 argued.

> ON THE EXAM
>
> An aggregate metric averages your best-served customers with your worst. Only one of those groups is complaining.

Proxy signals for quality

- Some problems have **no ground truth at all**, or truth that arrives years later.
- **Prediction distribution** is the first proxy: is the model predicting positive far more, or far less?
- **Confidence distribution** is the second: a model becoming uncertain is a signal.
- **Downstream behaviour** is the best proxy: overrides, complaints, appeals, manual corrections.
- A rising override rate is humans telling you the model is wrong, without anyone labelling anything.

> ON THE EXAM
>
> Human override rate is the most underused monitoring signal in production ML. It is free, and it is a real quality measure.

Monitor the business outcome too

- Model metrics answer whether the model is accurate. They do not answer whether it is **working**.
- Track the **business metric the model exists to move**: fraud losses, conversion, cost per case.
- A model can hold its accuracy while the business outcome degrades, if the population changed.
- That combination is worth investigating: technically fine, commercially worse.
- Module 17's principle: the right metric moves when the business outcome moves.

> ON THE EXAM
>
> The stakeholder never asked for accuracy. They asked for fewer fraud losses, and that is the number that should be on the dashboard.

Champion and challenger in production

- Keep a **challenger** model running alongside the production champion, as a shadow variant.
- You then have a **live comparison** rather than a one-off evaluation months ago.
- When the challenger consistently wins on real traffic, promotion is evidenced rather than argued.
- Module 24's shadow testing, left running rather than used once.
- It costs a second set of instances, continuously. That is the price of the comparison.

> ON THE EXAM
>
> Drift monitoring tells you the champion got worse. Champion-challenger tells you something else is now better, which is a different and more useful fact.

Monitoring without Model Monitor

- **Data capture** from module 20 still works. The production data is still recorded.
- Distribution comparisons are **published statistics**: chi-squared, KS, JS divergence. Compute them yourself.
- Run them on a schedule in a **Processing job** or a pipeline, from module 28.
- Log results to **SageMaker AI MLflow**, so the trend is visible over time. Module 19.
- Emit a custom CloudWatch metric, and the alarm and trigger path is unchanged.

> ON THE EXAM
>
> The same shape as Clarify's replacement. The service is closed; the statistics, the capture and the alarm path are all still there.

When monitoring is a requirement, not a practice

- Regulated models often carry an explicit **ongoing monitoring obligation**, not just a build-time one.
- That typically means **documented** monitoring, recorded results, and evidence of action taken.
- **Bias monitoring over time** is frequently required, not optional. Module 08's obligations continue after deployment.
- Model cards from module 19 are where the monitoring plan and its results are recorded.
- 'We would have noticed' is not evidence. A dated report is.

> ON THE EXAM
>
> For a regulated model, monitoring is part of the licence to operate. Switching it off is a compliance decision, not a cost saving.

Retiring a model

- Some models should be **switched off** rather than retrained, and that is a legitimate outcome.
- The business process changed, the product was discontinued, or a simpler rule now performs as well.
- An endpoint nobody has called in six months is **cost and risk** with no benefit.
- Retire deliberately: archive the model version, record why, and delete the endpoint.
- Undocumented retirement is how an endpoint quietly bills for two years.

> ON THE EXAM
>
> Monitoring tells you a model is failing. It is also how you discover a model nobody uses any more.

What monitoring costs

ITEM	COST SHAPE
Data capture	S3 storage for captured requests and responses
Baselining job	A one-off processing job per baseline
Each monitoring execution	A processing job, every schedule interval, forever
Frequent schedules	Hourly monitoring is 24 processing jobs a day, per monitor
Four monitors	Four times the above, if you enable all of them
Not monitoring	Free, until a degraded model has been wrong for a month

ON THE EXAM

Cadence times monitor count is the multiplier. Hourly across all four monitors is 96 processing jobs a day on one endpoint.

Monitoring at scale

- One endpoint with four monitors is manageable. **Fifty endpoints is a platform problem.**
- Standardise: the same monitors, the same cadence, the same alarm shape, defined in code.
- **Aggregate the alarms**, so one dashboard shows which models are healthy rather than fifty.
- SageMaker Projects from module 28 is how that standard gets applied to every new model.
- Per-model bespoke monitoring does not survive the tenth model.

> ON THE EXAM
>
> Monitoring designed for one model becomes unmanageable at scale. Design it as a platform capability from the third model onwards.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Monitor reports nothing at all	Data capture not enabled. Module 20
Model quality monitor finds no data	Ground truth offsets not set, so the merge matches nothing
Violations on every execution	Constraints too tight, or the baseline is stale
No violations ever, for months	Constraints too loose, or the baseline came from the wrong data
Cannot monitor image model inputs	Tabular data only. Monitor the output instead
Accuracy fell, no drift detected	Concept drift. Only model quality monitoring sees it
Monitoring cost surprising	Cadence times number of monitors. Each execution is a job

ON THE EXAM

Rows three and four are a matched pair, and the second is more dangerous: a monitor that never fires looks exactly like one that is working.

Monitor the **inputs** from day one. Monitor the **outcomes** as soon as labels exist.

> A demand forecasting model has just been deployed. Ground truth becomes available four weeks after each prediction, so no accuracy measurement is possible yet.
>
> The team wants to detect problems before those four weeks elapse, particularly upstream pipeline faults and shifts in the input data.

What should they enable now?

A Model quality monitoring, with a four-week offset

B Bias drift monitoring on the forecast outputs

C Data quality monitoring, with a baseline from the training data

D Nothing yet. Wait until ground truth is available

◆ PAUSE AND DECIDE

What should they enable now?

A Model quality monitoring, with a four-week offset

Worth setting up for later, and it cannot report anything for four weeks. It does not meet the stated requirement of detecting problems sooner.

B Bias drift monitoring on the forecast outputs

Needs ground truth labels, so it has the same four-week delay. Bias is also not what the team said they were worried about.

C Data quality monitoring, with a baseline from the training data

Needs inputs only, so it reports from the first execution. It catches both upstream faults and input shifts, which are exactly the stated concerns.

D Nothing yet. Wait until ground truth is available

Four weeks of unmonitored production. A broken upstream pipeline would run undetected for a month.

THE SIGNAL

Two of the four monitors need no labels. On a model with delayed ground truth, those two are everything you have for the first month.

A lending team enables model quality monitoring. Loan outcomes are known roughly 90 days after a decision, and they store those labels in S3 as they arrive.

The monitor runs daily and reports no data. The captured inference records are present, and the ground truth files are present.

What is wrong?

A Offsets are not configured, so the merge job cannot match labels to predictions

B Model quality monitoring does not support a 90-day label delay

C Data capture is disabled on the endpoint

D The baseline was created from the wrong dataset

◆ PAUSE AND DECIDE

What is wrong?

A Offsets are not configured, so the merge job cannot match labels to predictions

Model quality merges captured inference data with labels. With no offset it looks for labels alongside recent predictions, and those are 90 days away.

B Model quality monitoring does not support a 90-day label delay

Offsets exist precisely because ground truth is delayed. AWS documents them for this situation.

C Data capture is disabled on the endpoint

The scenario states captured inference records are present, so capture is working.

D The baseline was created from the wrong dataset

A wrong baseline produces wrong violations, not an empty result. The monitor would still report something.

THE SIGNAL

Both halves of the data are present and they are 90 days apart. The offset is what tells the merge job how far back to look.

> A manufacturer runs an image classification model detecting surface defects from production-line photographs. They want to detect when the incoming images start to differ from the training set, for example after a lighting change.
>
> An engineer proposes enabling data quality monitoring on the endpoint inputs.

What should you tell them?

A Agree. Data quality monitoring profiles any input type

B Agree, but use bias drift monitoring instead for image data

C Reject it. Model Monitor cannot be used with image models at all

D Model Monitor handles tabular data only. Monitor the output distribution, or embed the images and profile the vectors

◆ PAUSE AND DECIDE

What should you tell them?

A Agree. Data quality monitoring profiles any input type

AWS states Model Monitor computes metrics and statistics on tabular data only. There is no tabular distribution in a raw image to profile.

B Agree, but use bias drift monitoring instead for image data

Bias drift measures fairness metrics and needs labels. It is a different question entirely, and the tabular limitation still applies.

C Reject it. Model Monitor cannot be used with image models at all

Too strong. AWS says an image model can still be monitored - on its output, where there is a class distribution to profile.

D Model Monitor handles tabular data only. Monitor the output distribution, or embed the images and profile the vectors

The documented position, plus the practical route: predicted class distribution and confidence are tabular, and embeddings make image drift measurable.

THE SIGNAL

C and D differ on precision. The input cannot be monitored; the output can. Getting that half-right is not the same as getting it right.

> A fraud model's precision and recall have both fallen noticeably over six weeks. Data quality monitoring shows no violations at all: every input feature's distribution matches the baseline closely.
>
> A team member concludes that since there is no data drift, the model is fine and the metric calculation must be faulty.

How should you respond?

A Agree. No data drift means the model cannot have degraded

B This is the signature of concept drift. The inputs are unchanged; what they mean has changed

C Re-baseline data quality monitoring, since the baseline must be wrong

D Enable feature attribution drift monitoring instead of model quality monitoring

◆ PAUSE AND DECIDE

How should you respond?

A. Agree. No data drift means the model cannot have degraded

The reasoning is backwards. Data quality monitoring watches inputs and is structurally blind to a change in the relationship between inputs and outcome.

B. This is the signature of concept drift. The inputs are unchanged; what they mean has changed

Falling accuracy with unchanged input distributions is the definition. Fraud is the standard case: attackers change tactics, so old signals stop meaning fraud.

C. Re-baseline data quality monitoring, since the baseline must be wrong

The baseline is working correctly: the inputs genuinely have not drifted. Re-baselining would confirm the same thing.

D. Enable feature attribution drift monitoring instead of model quality monitoring

A useful addition and not a replacement. Attribution drift is an early warning; model quality is what already told them accuracy fell.

THE SIGNAL

The absence of data drift is evidence for concept drift, not against degradation. That inference is the wrong way round, and it is common.

What to take from this module

- **Data drift moves inputs. Concept drift moves meaning.** Only the second needs labels to see.
- **No data drift does not mean no degradation.** It is evidence for concept drift, not against it.
- **Two monitors need labels, two do not.** Start with the two that do not.
- **Ground truth is delayed, so set offsets**, and accept it as your detection floor.
- **Model Monitor is tabular only.** Image and text models are monitored on their output.

> ON THE EXAM
>
> Module 30 asks the same questions about foundation models and agents, where there is often no accuracy metric at all.